

Back-end em python



Prof^a. Ma. Ana Clara Gomes

SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Aula 14 — Herança e Polimorfismo

Objetivos da Aula

- Entender o conceito de **herança** em Python.
- Aprender como criar **classes filhas** que herdam atributos e métodos da classe pai.
- Compreender o **polimorfismo**: métodos com mesmo nome, comportamento diferente.
- Aplicar os conceitos em **projetos práticos**, consolidando POO.





Aula 14 — Herança e Polimorfismo



Herança

- Permite que uma **classe filha** reutilize **atributos e métodos** de uma **classe pai**.
- Evita repetição de código e facilita manutenção.

 Analogia:

Uma **classe pai** é como uma **receita básica de bolo**, e a **classe filha** é uma **variação da receita** (ex.: bolo de chocolate, bolo de cenoura).



Aula 14 — Herança e Polimorfismo



Polimorfismo

Permite que **métodos com o mesmo nome** se comportem de forma **diferente** dependendo do objeto.

Em Python, polimorfismo acontece com:

- Sobrescrita de métodos (**overriding**)
- Funções que aceitam diferentes tipos de objetos

 Analogia:

Um **botão de “ligar”** pode ligar o **carro**, a **máquina de café** ou o **computador** — mesma ação, resultados diferentes.



Aula 14 — Herança e Polimorfismo

Herança Simples

```
#herança simples
class Animal:
    def __init__(self, nome):
        self.nome = nome

    def som(self):
        print("Som genérico de animal")

class Cachorro(Animal):
    def som(self):
        print("Au Au!")

class Gato(Animal):
    def som(self):
        print("Miau!")

c1 = Cachorro("Rex")
c2 = Gato("Mimi")

c1.som()
c2.som()
```





Aula 14 — Herança e Polimorfismo



Polimorfismo

```
#polimorfismo em ação
def emitir_som(animal):
    animal.som()

animais = [Cachorro("Rex"), Gato("Mimi"), Animal("Genérico")]
for a in animais:
    emitir_som(a)
```



Aula 14 — Herança e Polimorfismo

Herança com super()

```
#Herança com super()

class Funcionario:
    def __init__(self, nome, salario):
        self.nome = nome
        self.salario = salario

    def mostrar_info(self):
        print(f"Nome: {self.nome}, Salário: {self.salario}")

class Gerente(Funcionario):
    def __init__(self, nome, salario, setor):
        super().__init__(nome, salario)
        self.setor = setor

    def mostrar_info(self):
        super().mostrar_info()
        print(f"Setor: {self.setor}")

g = Gerente("Alice", 5000, "TI")
g.mostrar_info()
```





Aula 14 — Herança e Polimorfismo



Boas Práticas

- Use **super()** para reaproveitar código da classe pai.
- Sobrescreva métodos apenas quando necessário.
- Pense em polimorfismo como “**mesma interface, diferentes implementações**”.
- Herança → código mais limpo e reutilizável.



Aula 14 — Herança e Polimorfismo



Aplicações no Mercado

- Sistemas bancários: contas diferentes com regras diferentes.
- Jogos: personagens com classes diferentes (guerreiro, mago, arqueiro).
- Aplicações web: diferentes tipos de usuários (admin, cliente, fornecedor).
- Frameworks de software: criar componentes reutilizáveis e extensíveis.

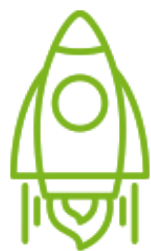


Aula 14 — Herança e Polimorfismo



Exercícios para Sala

1. Crie uma classe `Veiculo` com atributos `marca` e `modelo` e método `ligar()`.
 - Crie `Carro` e `Moto` que herdam de `Veiculo` e sobrescrevem `ligar()` com mensagens específicas.
2. Crie uma classe `Funcionario` com método `calcular_salario()`.
 - Crie `Gerente` e `Vendedor` que implementam `calcular_salario()` de forma diferente (polimorfismo).
3. Expanda o **simulador bancário**:
 - Adicione métodos exclusivos para cada tipo de conta.
 - Crie uma função `mostrar_extrato(conta)` que funcione para qualquer conta (polimorfismo).
4. Desafio: crie uma lista de animais diferentes e use uma função para **emitir som de todos**, mostrando polimorfismo.



Aula 14 — Herança e Polimorfismo



Alguma dúvida ?

clara.gomes@ufpe.br



SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO